

PLAGUE VILLAGE

A GAME OF SURVIVAL, STRATEGY, & CONSEQUENCES

Programming Capstone Project

Team #6

Janay Snell
Gavin McKenzie
Zoe Craig
Oscar Salinas-Villarreal
Maria Del Carmen Caballero Hernandez
Andrew Thanh Dang
Noah Yarosz

MEET THE TEAM



JANAY (PM)



ANDREW (BACKEND API, SAVE/LOAD)



GAVIN (TESTING, VIDEO CREATION/EDITING)



OSCAR (LOGIN/PROFILE)



NOAH (RATS/VILLAGERS)

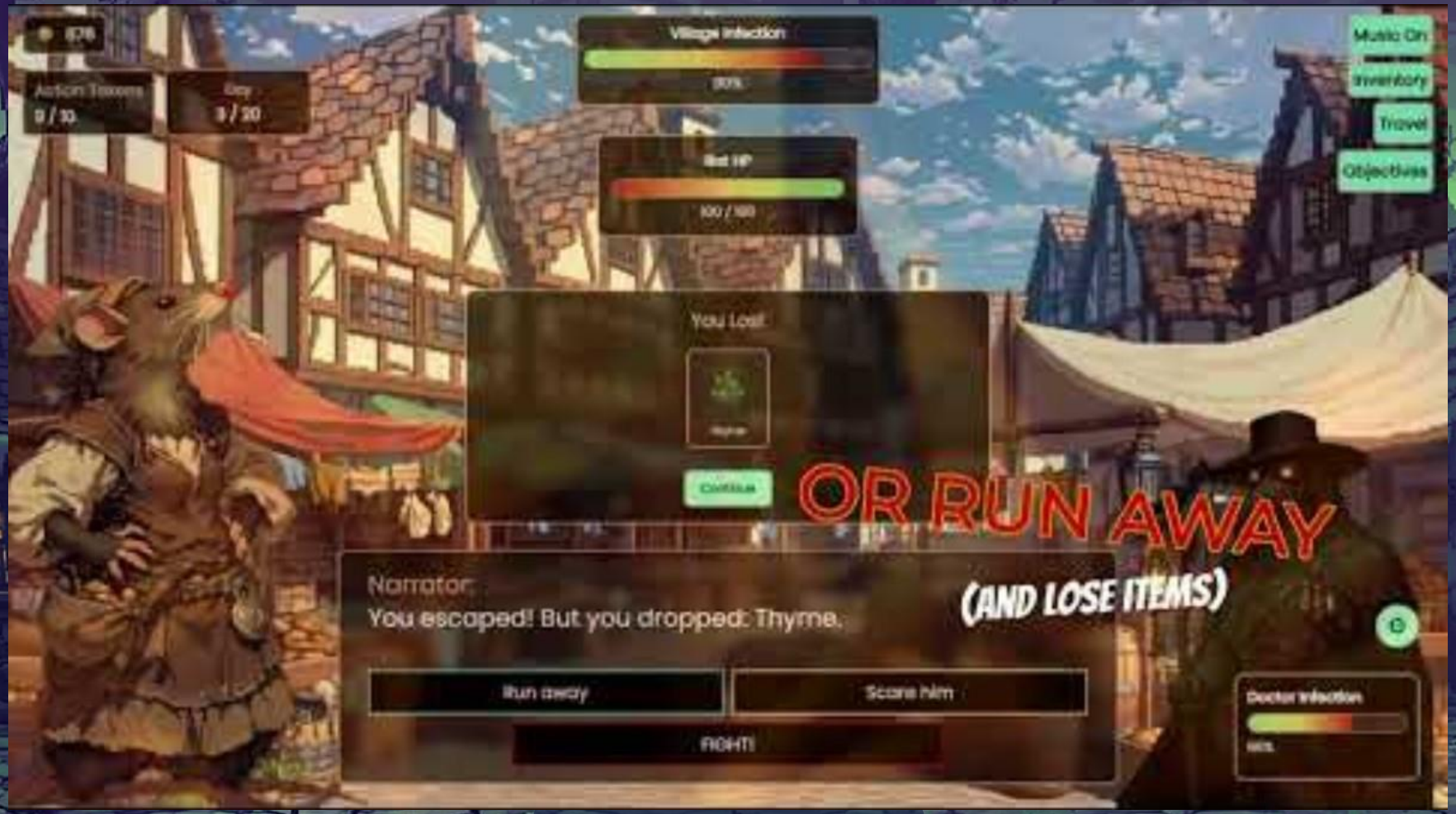


ZOE (FRONTEND LEAD, POTIONS/SHOP)



MARIA (FOREST/CHAPEL)

GAME TRAILER



Edited by: Gavin

The year is 1349...

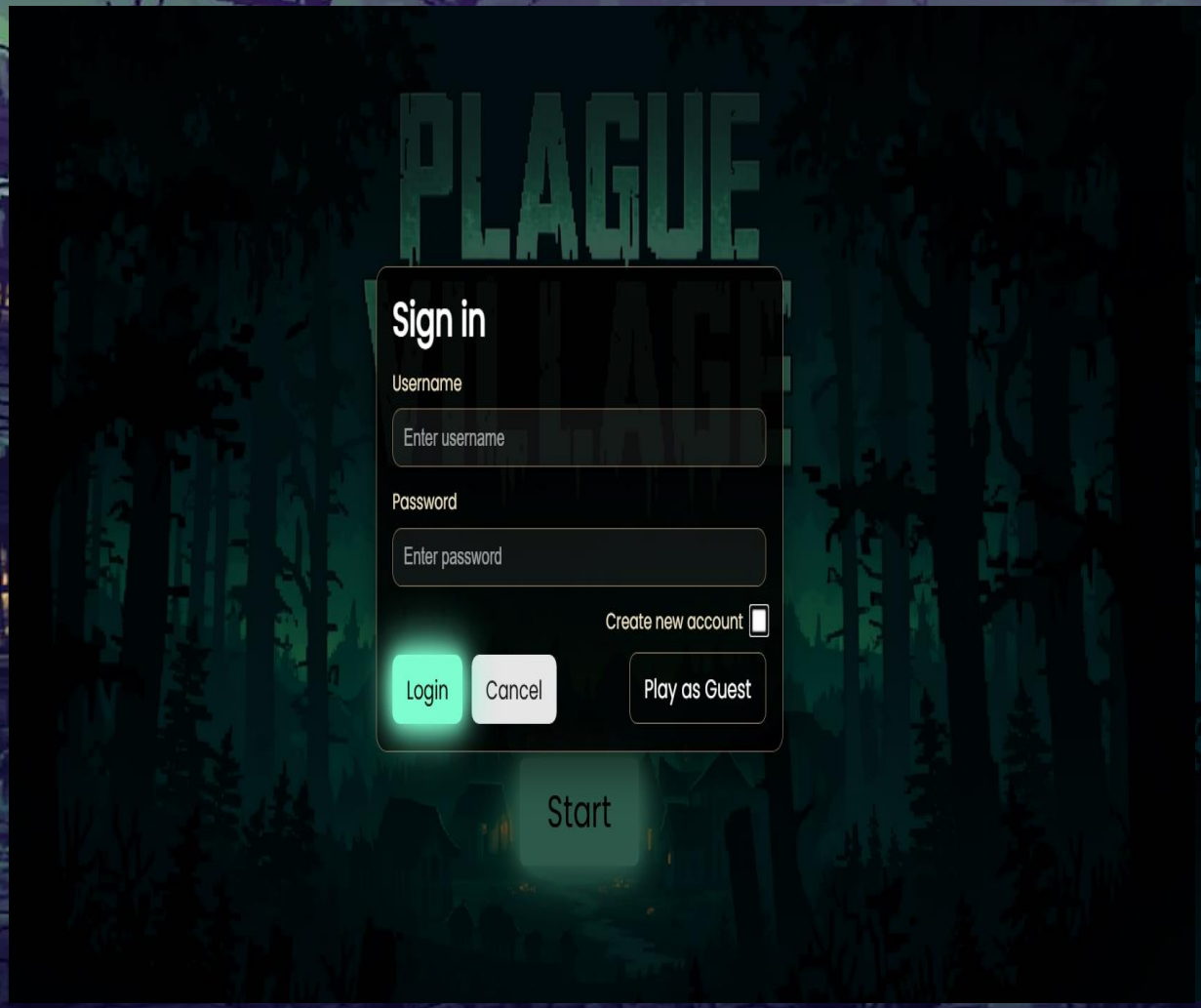
The player takes on the role of a traveling plague doctor who arrives in a dying medieval village. Rats spread disease through the streets, villagers fall ill, and time is limited. The player must gather ingredients, brew potions, battle rats, and decide who to treat, knowing that every choice affects survival outcomes...

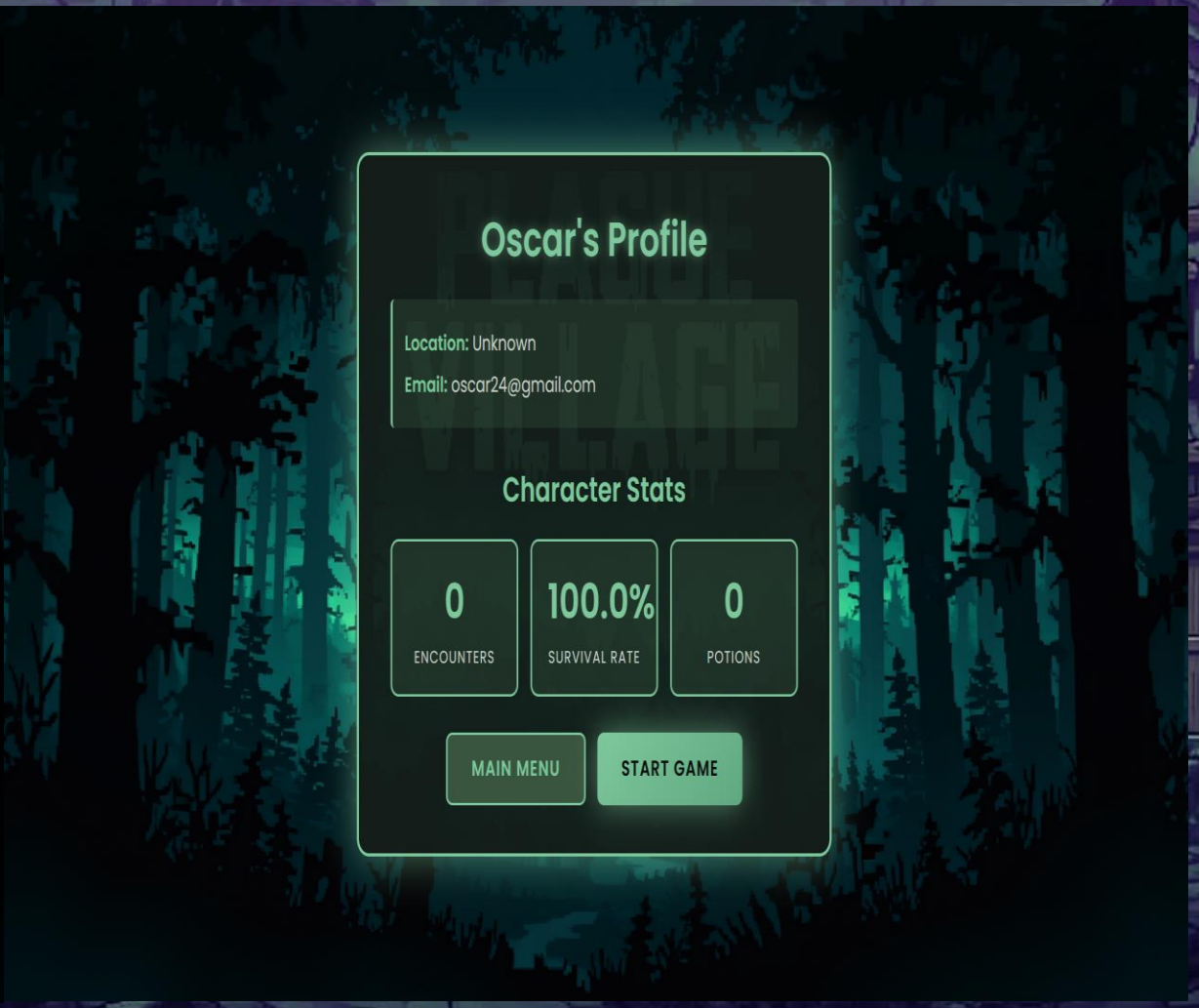


Project Requirements

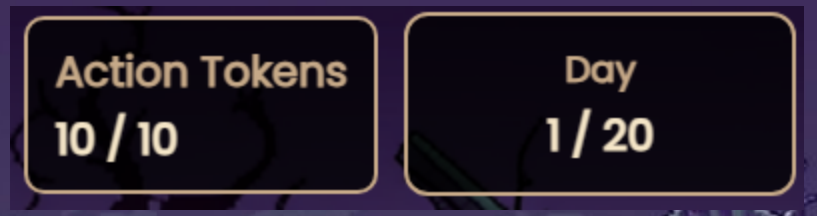
- User can create unique profiles
- User can log in
- User can load & save their game
- User can complete a full playthrough
- Each scene is well-designed & contributes to overall gameplay loop
- User must use strategy & resource management to win the game
- Outcomes are based on chance, so survival is not guaranteed
- User's choices have consequences, with 3 unique endings

Player Login & Profile Creation





Game Objectives & Core Gameplay Loop



- Main Objective: Defeat the rats & heal the villagers before Days run out
- Player receives 5 randomized inventory items & a randomized amount of money at the beginning of each playthrough to get them started
- From there, player must use strategy & resource management to make choices
- 20 days
- Each day has 10 Action Tokens
- Important actions cost Action Tokens, when AT's run out the player returns to the main village for the EOD Report
- Keep an eye on Doctor Infection to keep it from reaching 100





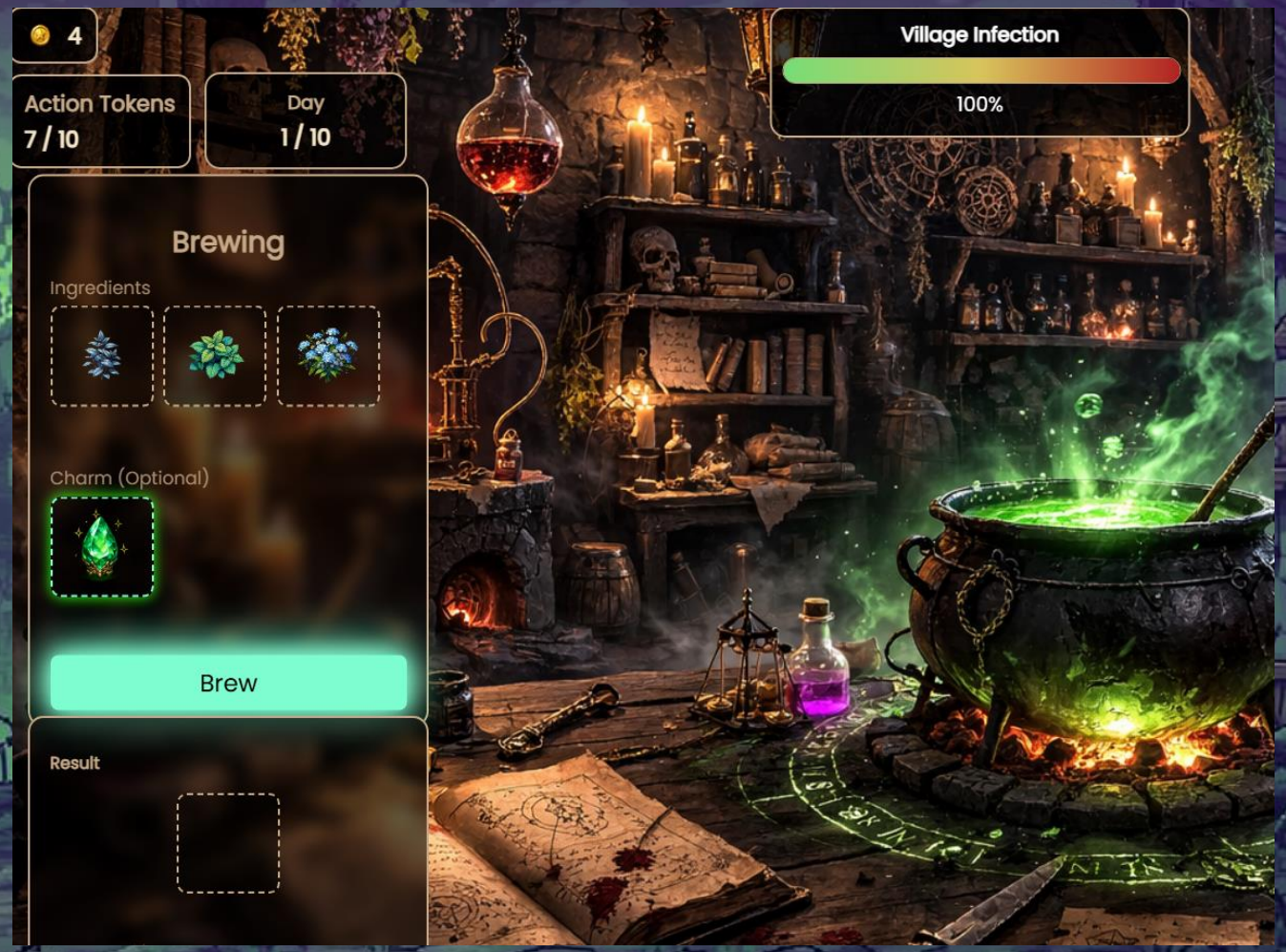
Forest



- Collect ingredients for potion brewing
- Multi-depth forest exploration
- Costs 1 action token
- Randomized ingredient discovery
- Variable outcomes

Potions

- Use ingredients from forest to brew potions
- Use charm items to boost potions, as seen here with the Brilliant Emerald (which we will go over later)
- Healing Tonic - Heals villagers
- Fever Suppressant – Protects villagers from infection increase for 1 day
- Plague concoction – Stops rats from growing stronger for 1 day
- Elixir – Heals Doctor infection
- Effectiveness Tiers – Chance based (weak, mid, strong)



Healing Villagers



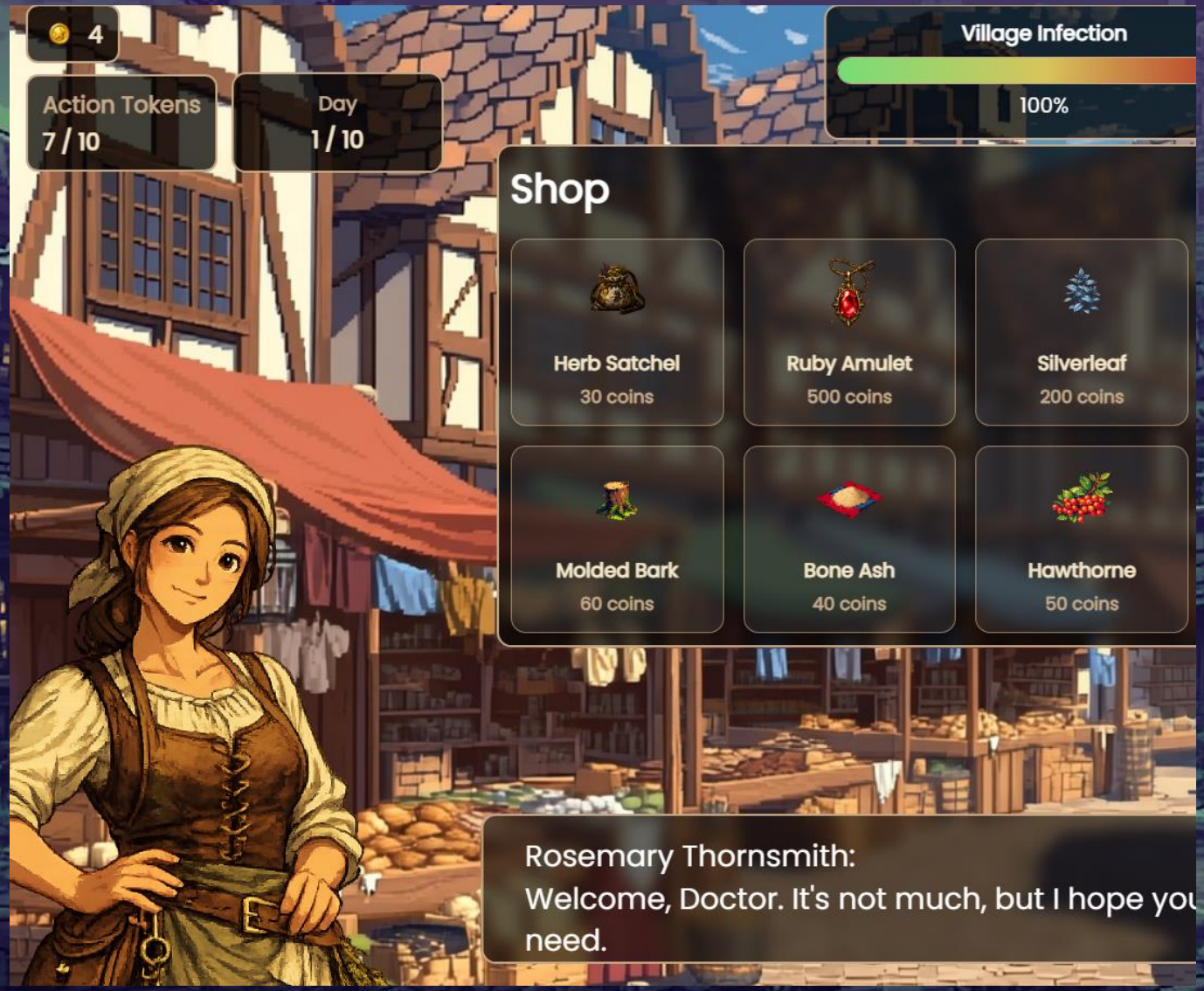
- 10 unique villagers with story
- Randomizes villagers every run
- Villager images change from sick to healed
- Use healing tonics & fever suppressants
- Ruby Amulet (protects villager permanently)
- When villager dies, villager turns grey

Chapel

- Pray for Dead or Living
- Costs 2 action tokens
- Timed prayer event (5 seconds)
- Chance based outcomes
- Dead → possible revival of villager
- Living → possible infection reduction



Shop



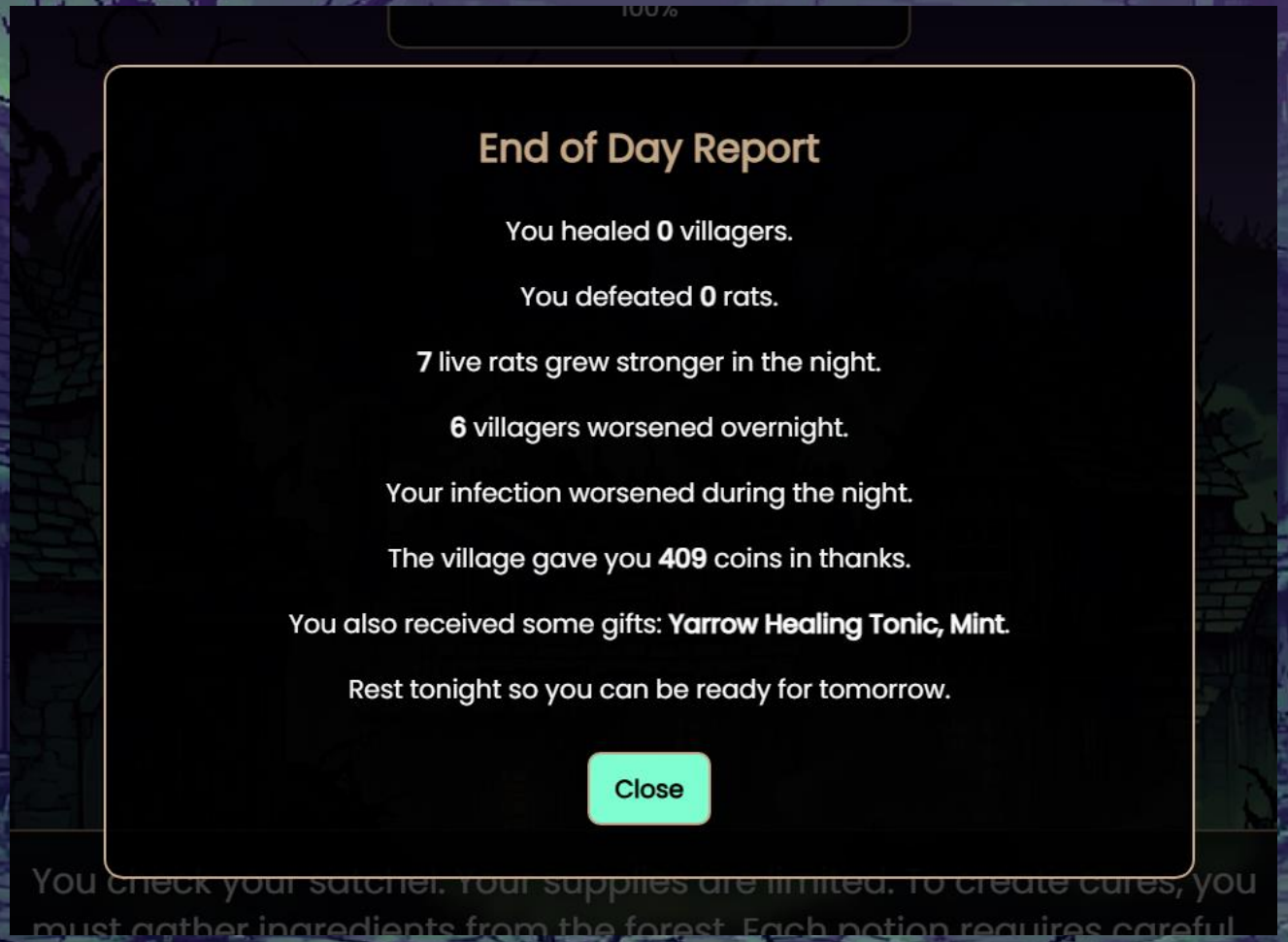
- Shop's contents are randomized & the shop restocks daily
- Boost items which help with actions in other scenes
- Ruby Amulet (Protects villagers – like the fever suppressant, but lasts entire game)
- Herb Satchel (Protects Doctor)
- Final Vigil Candle (boosts revival prayers)
- Rosemary Bundle (boosts healing prayers)
- Brilliant Emerald (boosts potions – increases strength of healing tonics & elixirs)
- Also carries a few ingredients

Rat Battles

- Attack: Risk of infection
- Scare: Rats drop items / Risk of infection
- Run Away: You drop items
- Concoctions: Bonus Damage
- Dialogue: Skips from reentering
- Upon Defeat: Gain gold



End of Day Report



- Player receives EOD report when they run out of AT's & return to the main village
- How many villagers healed/rats killed
- How many villager infections got worse/Did Doctor infection get worse
- How many rats got stronger
- Money/gifts from the village

Game Endings (Bad)





- Doctor Dies Ending, triggered by Doctor infection reaching 100%
- Village Collapse Ending, triggered by failure to meet the objectives by the 20-day limit

Game Ending (Good)



- Village Saved Ending
- You win!!!

System Architecture & Deployment

- Architecture: The software follows a layered structure with a presentation layer for the user interface, a game logic layer for mechanics and scene behavior, and a game state layer for tracking progress, inventory, and objectives.
- Design Principles: The system was built using modularity, separation of concerns, and maintainability, making it easier to update scenes and core systems independently.
- Requirements: The game requires Python, Flask, and a modern web browser.
- Deployment: The game is currently deployed as a local web application.

Testing & QA

- Save/Storage
- Cryptography
- Summary



Development Process & Challenges

Planning & Concept Design

- Defined game idea, theme, and core mechanics (survival, decision-making)
- Created initial wireframes and story flow

Technology Stack

- Frontend: HTML, CSS, JavaScript
- Backend: Python
- Tools: GitHub, VS Code, testing tools

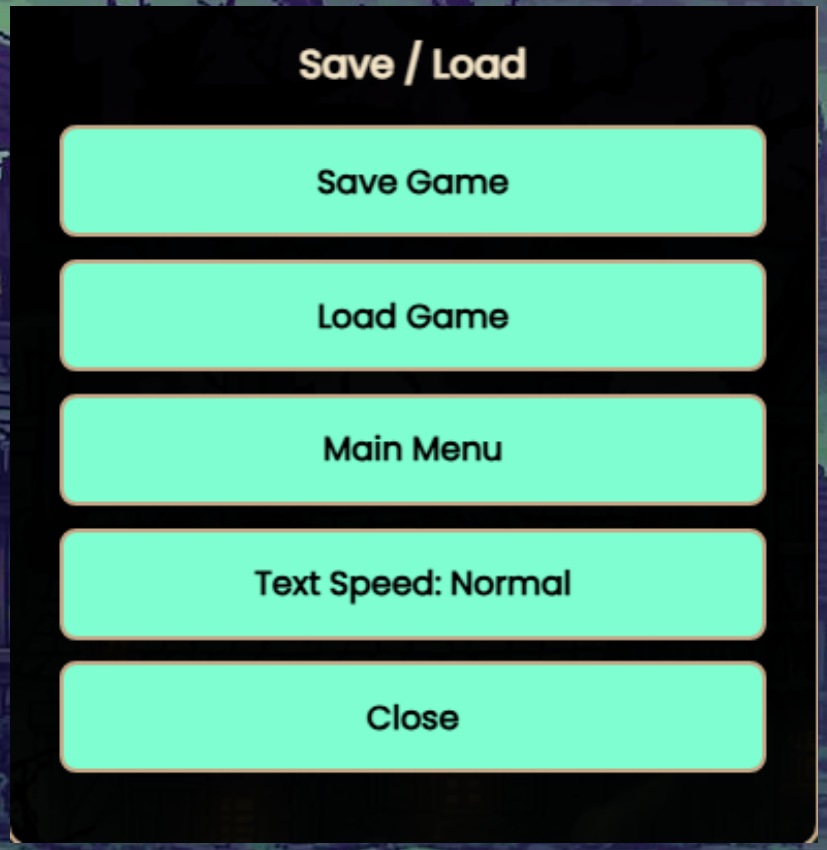
Iterative Development

- Built core features first (navigation, choices, game logic)
- Incrementally added UI improvements and new features
- Used version control for collaboration and tracking changes

Testing & Debugging

- Unit testing for logic
- UI testing for user interactions
- Bug fixing and performance improvements

User Feedback



Changes Made Based on User Feedback

- Play testers felt some dialogue was too slow
  - We added a menu option to change the text speed
- Players wanted a way to return to the main menu without having to reload the page
  - We added a return to main menu button
- Some Users complained about the travel to villager house menu was hard to read
  - We added a background and changed the color of the text to make it more readable
- Some users had issue with the volume of the potion creation sound effect
  - We lowered the sound effect volume

Challenges

Scheduling Conflicts

- Difficulty finding consistent meeting times slowed progress

Communication Gaps

- Misalignment on tasks and expectations early in the project

Unclear Initial Vision

- Needed more time to define goals, features, and scope

Project Scope & Scaling

- Balancing ambition with a realistic, achievable project size

Delayed Role Assignment

- Late decision on Project Manager caused an initial slowdown

Incomplete Features

- Time constraints prevented full implementation of all planned ideas

Lessons Learned

Scheduling Conflicts

- Establish clear roles early (especially Project Manager)

Communication Gaps

- Set regular meeting times from the start

Unclear Initial Vision

- Prioritize strong communication and check-ins

Project Scope & Scaling

- Define project scope early to avoid over-expanding

Delayed Role Assignment

- Focus on a minimum viable product (MVP) first

Incomplete Features

- It's better to fully complete core features than partially build many

Future Plans

- Add **more story branches and endings**
- Mobile-friendly App
- Multiplayer or shared decision gameplay


THANK YOU